Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

SECURE MARKSHEET VERIFIER: Tamper-Evident Verification

Team Name: *Team #*	*The Verifiers*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Rawat, Diksha – 2510371956* *rawat.diksha2067@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Gupta, Aakhya – 2510380033* *aakhyasuhani@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Singh, Divya – 2510370275* *ddivyasingh2006.ds@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Srivastava, Jeevika – 2510370298* *jeevikasrivastava@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract

The problem with the existence of fake or manipulated physical marksheets arises as there is often no quick way for the recruiters, Human Resources personnel, scholars, etc., who are from outside sources, to authenticate them. Giving access to the complete university database containing all student information will lead to compromising other students' record.

The Secure Marksheet Verifier solves this problem by using a Merkle Hash Tree in combination with Optical Character Recognition (OCR) of photos of physical mark sheet. Certified students' records are first hashed by university through a specially designed dual-hash function on the basis of FNV-1a and DJB2 functions. A Merkle tree will be constructed starting from the bottom upwards. Only Merkle Root Hash of all documents will be released to the public domain thereby ensuring privacy of student records.

To verify the document, an outside party captures an image of the document. This OCR program captures identifiers from the documents like roll number and subject marks. These captured identifiers will be hashed by verification engine and will be checked against the Merkle Root Hash released in public through Merkle Proof with logarithmic O(log N) time complexity. If the proof fails, the record will be audited in detail to find which specific subject or value is modified. Due to the avalanche effect of the hash value generation, if any digit is altered in a particular mark of any subject, a completely new hash is generated. Students' records, Verification Log, and Audit Results will be saved in CSV file format.

Implementation will use C++ Language along with Standard Libraries. The project uses the data structure concepts like Merkle Trees, Hash tables, Vectors, Recursion, Custom Hash, and the object-oriented concepts like Classes, Encapsulation, Abstraction, Structures, File I/O. Main modules of the projects are Custom Hash Engine, Merkle Trees Module, OCR Engine, and Verification Engine.

Main aim is to perform quick verification of physical mark sheets without compromising the privacy of all students' records.

Updated Project Approach and Architecture

Approach
A Merkle Tree is generated from certified student records at the university level and only the Merkle Root Hash is shared outside. For verifying the marksheet, a third party takes a photograph of the marksheet. The system parses the roll number and marks and verifies those fields against the publicly available root. In case of failure, the fields undergo individual auditing for determining the tampered subject and marks.
System Design
The system is a modular C++ program designed as shown below:
University Record → CustomHash → Merkle Tree → Marksheet Photo → Verification → Verdict
1. CustomHash Engine: Makes use of FNV-1a and DJB2 hashing algorithms for generation of hash values for the student records and for the leaf nodes of Merkle Tree.
2. Merkle Tree: Constructs the Merkle Tree using vectors in a bottom up manner recursively using vectors.
An unordered map :
helps in quick searching of records.
while Merkle proofs helps in verification process with O(log N) complexity.
3. OCR Engine: Extracts the roll number and marks for all subjects from the marksheet photo.
4. Verification Engine: Rehashes the roll numbers and marks to get the leaf hash value, which is verified against the publicly available root using Merkle proof. If there is a failure in verification, it compares individual fields to find the tampered one.
Data Flow and Communication
University record is hashed into the Merkle Tree and OCR extracts the necessary data from marksheet photo. The records, ledger and auditing data are stored in CSV format files. Since the application runs locally, no network protocols are required. The modules communicate through the interface provided by the C++ classes. Only the Merkle Root Hash is revealed publicly.
Libraries and Tools
Language: C++
Library: C++ Standard Library, including vectors, unordered$_map, and filestreams$
Algorithms: FNV-1a, DJB2, and bottom-up Merkle Tree construction
Version Control: GitHub

Tasks Completed

Task Completed	**Team Member**
Custom hash engine (FNV-1a and DJB2)	Diksha Rawat
Binary hash tree (Merkle Tree)	Diksha Rawat
Merkle proof generation and verification	Jeevika Srivastava
OCR engine	Jeevika Srivastava
Dataset prepared	Aakhya Gupta
Field-by-field tamper audit	Divya Singh
All code committed to GitHub	

Challenges/Roadblocks

1. Odd number of records: If there isn't a pair for the last node, duplication is done. The evaluation will take into account both even and odd numbers of total records.
2. Lack of collision resistance of non-cryptographic hashes: Although FNV-1a and DJB2 are faster, they are not collision-resistant. A dual-hash approach will be used and tested with small input variations.
3. Consistent formatting of the records: Spacing differences, order of fields or delimiters may lead to valid marksheets being rejected. A consistent record format will be used by both the university and the verifier.
4. OCR issues: Problems such as blur, poor illumination and look-alike characters (for example 0/O or 1/l) may introduce errors. Image pre-processing and simple field validation will be done.
5. Management of the proof path: Every Merkle proof must keep the correct left/right placement of all the siblings. Proofs will be tested using several leaf nodes.
6. Distinguishing OCR errors from tampering: Wrongly read digits may look like fabricated marks. Extracted values will be shown for manual verification and fields with low confidence will be flagged.
7. Module integration: With the involvement of four people working on different modules, interface specifications will be set up in advance. Version control (GitHub) and unit testing will help in easy integration.

Tasks Pending

Task Pending	**Team Member (to complete the task)**
CSV logging for records, ledger and audits	Diksha Rawat
Integrating all modules into one pipeline	Diksha Rawat
Improving OCR accuracy on real photos	Aakhya Gupta
Testing on the full dataset and unit-test results	Jeevika Srivastava
Final testing of tamper audit and edge cases	Jeevika Srivastava
Final project report	Divya Singh
Final presentation and viva demo preparation	All

Project Outcome/Deliverables

Major Deliverables
- Instant validation of the physical marksheets without exposing the entire database of the university through the exchange of just the Merkle Root Hash.
- Strong tamper detection as any change made even in a single digit of the marks will give totally different hashes (avalanche effect).
- Quick validation using O(log N) Merkle Proofs.
- Fine-grained auditing with identification of the specific subjects modified.

Deliverables
- C++ source code consisting of four modules namely CustomHash Engine (implementations of FNV-1a and DJB2), Merkle Tree Module, OCR Engine, and Verification Engine.
- CSV logs for student details, log of published roots, and audit reports.
- Unit test report of each module.
- Project report and project presentation.
- Complete running demo for viva voce.

Progress Overview

The project is well-advanced. The design process is complete, which includes proposal, goals, milestones, and architecture of the system. As far as implementation goes, the following elements have been implemented:
- Custom hash engines (FNV-1a and DJB2)
- Binary hash tree (Merkle Tree)
- Generation and validation of Merkle proofs
- Optical character recognition (OCR) engine
- Audit for field-by-field tampering

The test data is ready. All major modules – CustomHash Engine, Merkle Tree Module, OCR Engine, and Verification Engine – are implemented and this gives a degree of completeness of about 70-75%.

Future tasks are: to integrate all the modules into one end-to-end pipeline, create the CSV logs for records, ledger, and audits; perform a full-fledged test run on the test dataset; and write a final report with viva demonstration.

The schedule status shows no substantial lags. The main potential problem is the accuracy of OCR for real photographs, which needs more tests.

It can be seen from the progress that the core modules were completed ahead of schedule, which provides extra time for integration and testing.

Codebase Information

Repository link: https://github.com/Diksha-Rawat-2067/Secure-Marksheet-Verifier
Branch: main
Important commits:
Custom hash engine code (FNV-1a and DJB2)
Binary hash tree (Merkle Tree) code
Merkle proof generation and verification code
OCR engine code
Dataset for testing and verification
Field-by-field tamper audit code

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Custom hash unit test	Pass	Same input, same hash
Merkle proof generation and verification	Pass	Even and odd records
Tamper detection	Pass	Altered marks rejected
Field-by-field audit	Pass	Tampered subject found
OCR extraction	Pass	Roll no. and marks read
End-to-end integration	In progress	Modules being combined

Deliverables Progress

C++ Source Code (four modules): Completed.
CustomHash Engine (FNV-1a and DJB2): Completed and committed.
Merkle Tree module with proof generation and verification: Completed and committed.
OCR Engine: Completed and committed.
Verification engine with field-by-field tamper audit: Completed and committed.
Test Dataset: Completed. The test dataset is ready and committed in the repository.
CSV Logs (records, ledger, audits): In Progress. The I/O operations of the log files are under way and being integrated into the modules.
Unit-test results: In progress. Modules are being tested individually and results are yet to be compiled after testing on the entire test dataset.
Project report and Presentation: In Progress. Phase I presentation is ready, and the final project report will be done after integration and testing.
End-to-End Working Demonstration: Pending. This is the next step which is pending after integration and before viva.